

Day 43 — Line-by-Line Syntax Explanation

day43_real_images_three_blocks.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 7, Day 2 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula (per-channel normalization, the receptive-field recursion extended to three blocks, or a network's parameter count), a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system — including the three-block training work cycle, the pooling budget, and the script's own real output graphs.

MATH box legend: $\times$ = matrix multiply / elementwise times W, b = filter weights/bias m = batch size k = kernel size C_{in}/C_{out} = input/output channel count r, j = receptive field, jump

Segment 0 — Imports and Global Setup

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image
from sklearn.datasets import load_sample_images

np.random.seed(42)
```

Two new imports today: **PIL.Image** for real JPEG resizing (Topic 1), and **sklearn.datasets.load_sample_images**, which returns two ACTUAL photographs bundled inside scikit-learn itself — no network download needed, and no synthetic data involved for this part of the lesson.

Segment 1 — Shared Helpers: reused verbatim from Days 39-42

```
def relu(z):
    return np.maximum(0, z)

def tanh(z):
    return np.tanh(z)

def softmax(z):
    z_shifted = z - z.max(axis=1, keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / exp_z.sum(axis=1, keepdims=True)

def he_scale(n_in):
    return np.sqrt(2.0 / n_in)
```

relu/drelu/tanh/dtanh/softmax/one_hot/cross_entropy_loss/he_scale are all character-for-character identical to Day 42's versions — today's lesson changes what the input LOOKS like (real photos, 3 channels, a third block) and how it's processed, not these pointwise/loss primitives.

Segment 2 — conv2d_forward_mc / conv2d_backward_mc: reused verbatim

MATH

$$Z[:,f,i,j] = \text{sum over } c,a,b \text{ of } X[:,c,i+a,j+b] \times W[f,c,a,b] + b[f]$$

```
def conv2d_forward_mc(X, W, b):
    m, C_in, H, Win = X.shape
    C_out, C_in_w, kH, kW = W.shape
    out_H = H - kH + 1
    out_W = Win - kW + 1
    Z = np.zeros((m, C_out, out_H, out_W))
    for f in range(C_out):
        for i in range(out_H):
            for j in range(out_W):
                patch = X[:, :, i:i+kH, j:j+kW]
                Z[:, f, i, j] = np.sum(
                    patch * W[f], axis=(1, 2, 3)) + b[f]
    return Z
```

Not one character of this function changed from Day 42. It was already written to sum over an ARBITRARY number of input channels (**C_in**), so calling it with real 3-channel RGB data today required zero modification — the exact meaning of Topic 1's 'channel count changes how much a unit computes, not how far it can see.'

```
def conv2d_backward_mc(dZ, X, W):
    ...
    for f in range(C_out):
        for i in range(out_H):
            for j in range(out_W):
                patch = X[:, :, i:i+kH, j:j+kW]
                dZ_ij = dZ[:, f, i, j]
                dW[f] += np.sum(
                    dZ_ij[:,None,None,None]*patch, axis=0)/m
                db[f] += np.sum(dZ_ij) / m
                dX[:, :, i:i+kH, j:j+kW] += (
                    dZ_ij[:,None,None,None] * W[f])
    return dW, db, dX
```

Also unchanged. Today's Segment 9 gradient check on a three-block network's dWc1 is really re-confirming that THIS function's dX return value chains correctly through THREE stacked calls, not just two — the function itself needed no new code to support that.

Segment 3 — maxpool_forward/backward, avgpool_forward: reused

```
def maxpool_forward(A, size=2, stride=2):
    ...
    flat = window.reshape(m, C, size * size)
    idx = flat.argmax(axis=2)
    ...

def avgpool_forward(A, size=2, stride=2):
    ...
    out[:, :, i, j] = window.mean(axis=(2, 3))
    return out
```

maxpool_forward/maxpool_backward are copied over unchanged from Day 41/42 — pooling was already channel-count-agnostic. **avgpool_forward** is also unchanged from Day 42: still used ONLY for the receptive-field sensitivity sweep (Segment 6), never for actually training the three real networks.

Segment 4 — demo_real_preprocessing: real photos, resize, normalize

```
photos = load_sample_images()
china = photos.images[0]
flower = photos.images[1]
```

load_sample_images() returns a small object bundling TWO real (427, 640, 3) uint8 JPEG photographs that ship inside the scikit-learn package itself — **.images[0]** and **.images[1]** pull out each one as a plain NumPy array, no download or internet access required.

```
def resize(img_uint8, size):
    pil = Image.fromarray(img_uint8)
    pil = pil.resize((size, size), Image.LANCZOS)
    return np.asarray(pil).astype(np.float64)
```

Image.fromarray converts a NumPy array into a PIL image object; **.resize(..., Image.LANCZOS)** is PIL's high-quality resampling filter, the same kind torchvision's **transforms.Resize** uses under the hood; **np.asarray(...).astype(np.float64)** converts back to a NumPy array of floats, ready for the arithmetic that follows.

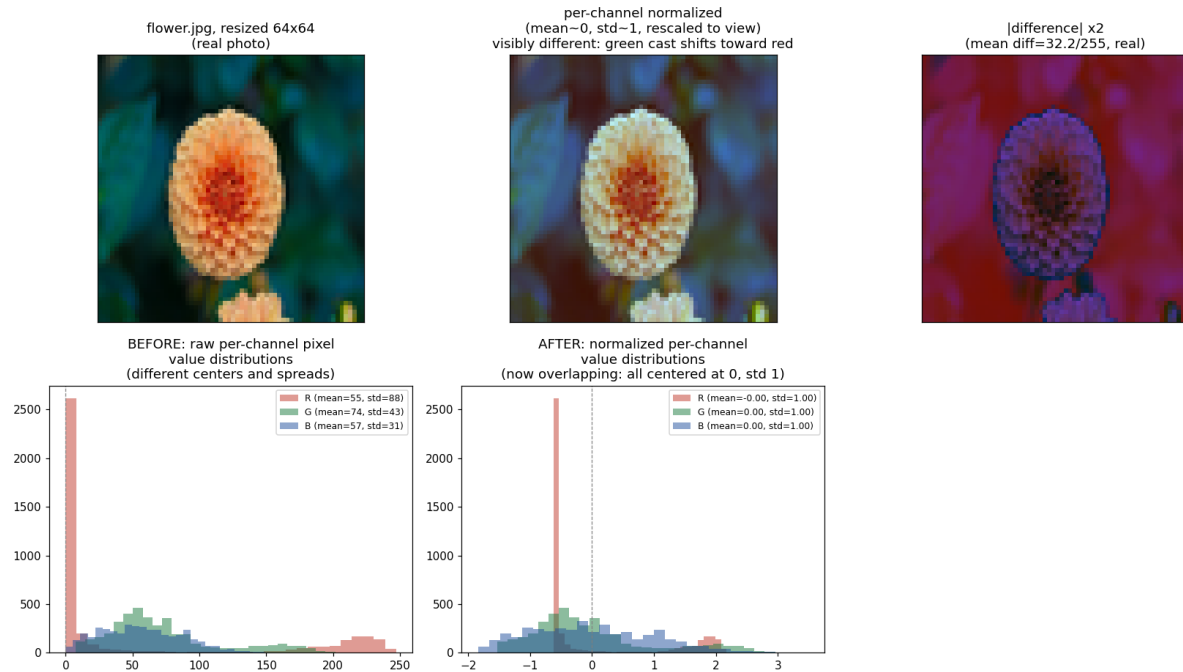
MATH

$$\text{mean}_c, \text{std}_c = \text{img[:, :, c].mean()}, \text{img[:, :, c].std()} \quad \text{for } c \text{ in } \{R, G, B\}$$

```
def normalize_channels(img):
    mean = img.mean(axis=(0, 1), keepdims=True)
    std = img.std(axis=(0, 1), keepdims=True)
    return (img - mean) / (std + 1e-8), mean.ravel(), std.ravel()
```

axis=(0, 1) averages over the HEIGHT and WIDTH axes ONLY, leaving the channel axis (axis 2) untouched — this is exactly what makes the result PER-CHANNEL: one mean and one std survive for each of R, G, B independently. **keepdims=True** keeps the result broadcastable against the original (H, W, 3) image directly.

REAL GRAPH (script's own output)



REAL GRAPH (script's own output) -- flower.jpg (chosen over china.jpg because its per-channel stds are far enough apart, 30.94 to 88.18, that normalization is visibly not just numerically different), its per-channel normalized version, an amplified difference map, and its real per-channel pixel histograms before and after normalization.

Segment 5 — demo_augmentation: flip, crop, brightness jitter

```
flipped = china_r[:, ::-1, :]  
diff_flip = np.mean(np.abs(flipped - china_r))
```

`[:, ::-1, :]` is NumPy's slice-with-negative-step syntax — it reverses the WIDTH axis (axis 1) while leaving height and channels alone, which is exactly a horizontal flip. **`np.mean(np.abs(...))`** then measures, with a single real number, how different the flipped image actually is from the original.

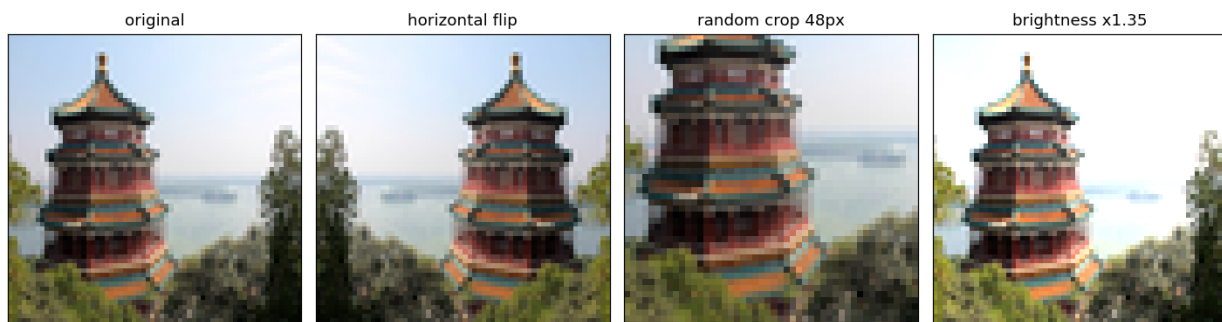
```
top = rng.randint(0, h - crop_size)  
left = rng.randint(0, w - crop_size)  
cropped = china_r[top:top+crop_size, left:left+crop_size, :]
```

`rng.randint(0, h - crop_size)` picks a random valid top-left corner so the crop never runs off the edge of the image; plain array slicing then extracts that sub-region — no special cropping function needed, just NumPy indexing.

```
jittered = np.clip(china_r * 1.35, 0, 255)  
changed_frac = np.mean(np.abs(jittered - china_r) > 1e-6)
```

`np.clip(..., 0, 255)` is essential here, not optional — multiplying by 1.35 can push pixel values above 255, which is not a valid pixel intensity, so clip enforces the valid range exactly the way a real image display or save-to-file step would require.

REAL GRAPH (script's own output)



REAL GRAPH (script's own output) -- the original photo and all three real augmentations applied to its actual pixels.

Segment 6 — receptive_field_growth extended, verified on 3 channels

```
one_block = [(3, 1), (2, 2)]
two_block = one_block + [(3, 1), (2, 2)]
three_block = two_block + [(3, 1), (2, 2)]
r3, j3 = receptive_field_growth(three_block)
```

receptive_field_growth itself is unchanged from Day 42 — extending it to three blocks is just list concatenation (**two_block + [...]**), appending one more (kernel, stride) pair for the loop to process. No new logic, only a longer input list.

```
Wc1 = rng.randn(3, 3, 3, 3); bc1 = np.zeros(3)
base_X = rng.randn(1, 3, size, size)

def forward_3block(X):
    p1 = avgpool_forward(conv2d_forward_mc(X, Wc1, bc1), 2, 2)
    p2 = avgpool_forward(conv2d_forward_mc(p1, Wc2, bc2), 2, 2)
    p3 = avgpool_forward(conv2d_forward_mc(p2, Wc3, bc3), 2, 2)
    return p3[0, 0, 0, 0]
```

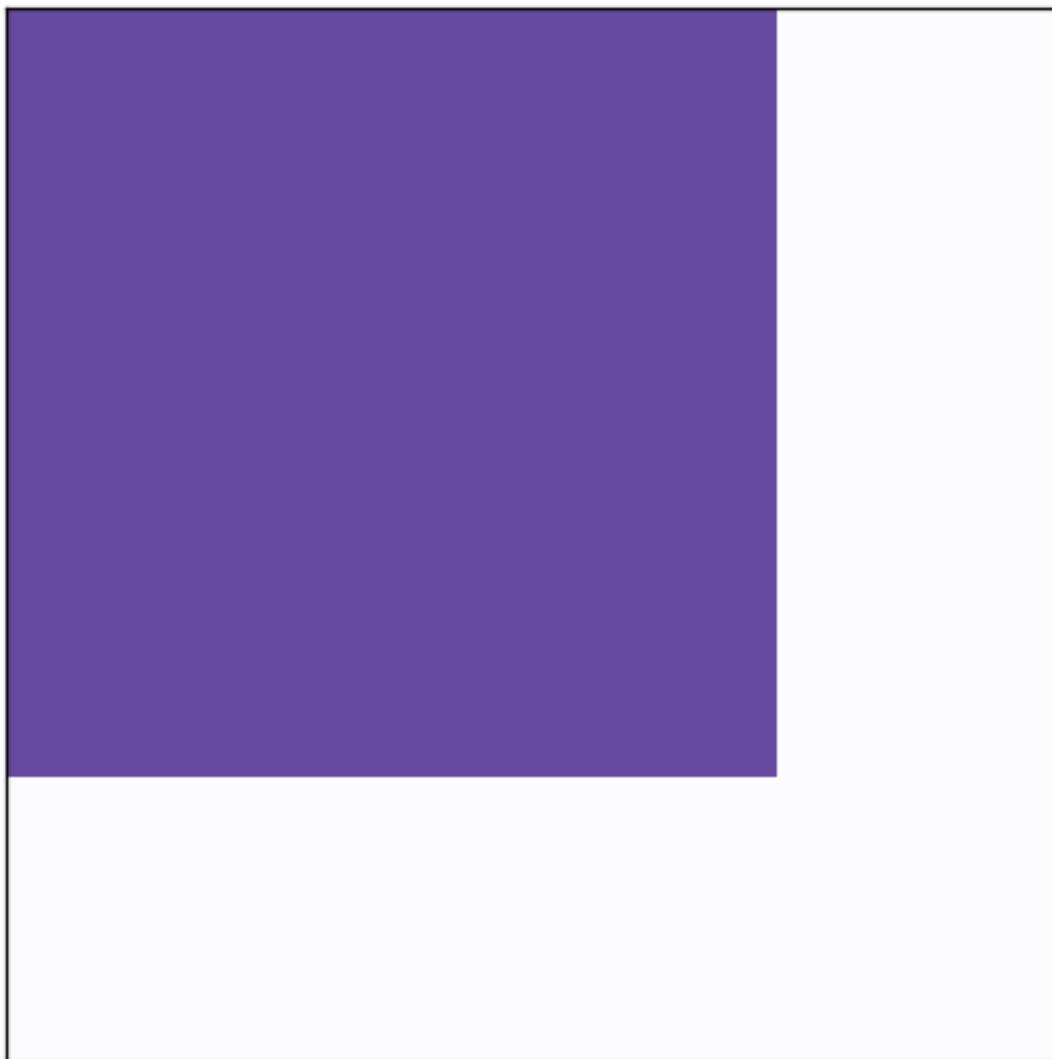
Wc1 = rng.randn(3, 3, 3, 3) — the SECOND 3 here is **C_in=3**, a genuinely 3-channel synthetic input this time (Day 42's equivalent check used C_in=1). Chaining three avgpool_forward/conv2d_forward_mc pairs and reading a single output value is the entire sensitivity-sweep target function.

```
sweep = 26
for r in range(sweep):
    for c in range(sweep):
        Xp = base_X.copy(); Xp[0, :, r, c] += eps
        if abs(forward_3block(Xp) - base) > 1e-9:
            sensitive.append((r, c))
```

Xp[0, :, r, c] += eps — the **:** in the channel position perturbs ALL 3 channels at pixel (r,c) together, treating a 'pixel' as one unit across channels. **sweep = 26** was widened from an initial (too small) 20 after the smaller window produced a false, clipped match — a real bug caught by comparing against the formula's independent 22x22 prediction.

REAL GRAPH (script's own output)

3-block receptive field: $22 \times 22 = 484$ px
on a 30×30 , 3-channel image



REAL GRAPH (script's own output) -- the corrected three-block receptive field, 22×22 , verified on a genuinely 3-channel input.

Segment 7 — make_junction_dataset_rgb

```
cx = cy = size // 2
r = cy + rng.randint(-center_jitter, center_jitter + 1)
c = cx + rng.randint(-center_jitter, center_jitter + 1)
```

The junction's center is now computed relative to the image's MIDDLE (**cx, cy**) plus a small random offset, instead of Day 42's **rng.randint(margin, size-margin)** which let it wander across the entire canvas. **center_jitter + 1** in the upper bound is needed because **randint**'s upper bound is exclusive — without the **+1**, the maximum possible jitter would never actually be reached.

MATH

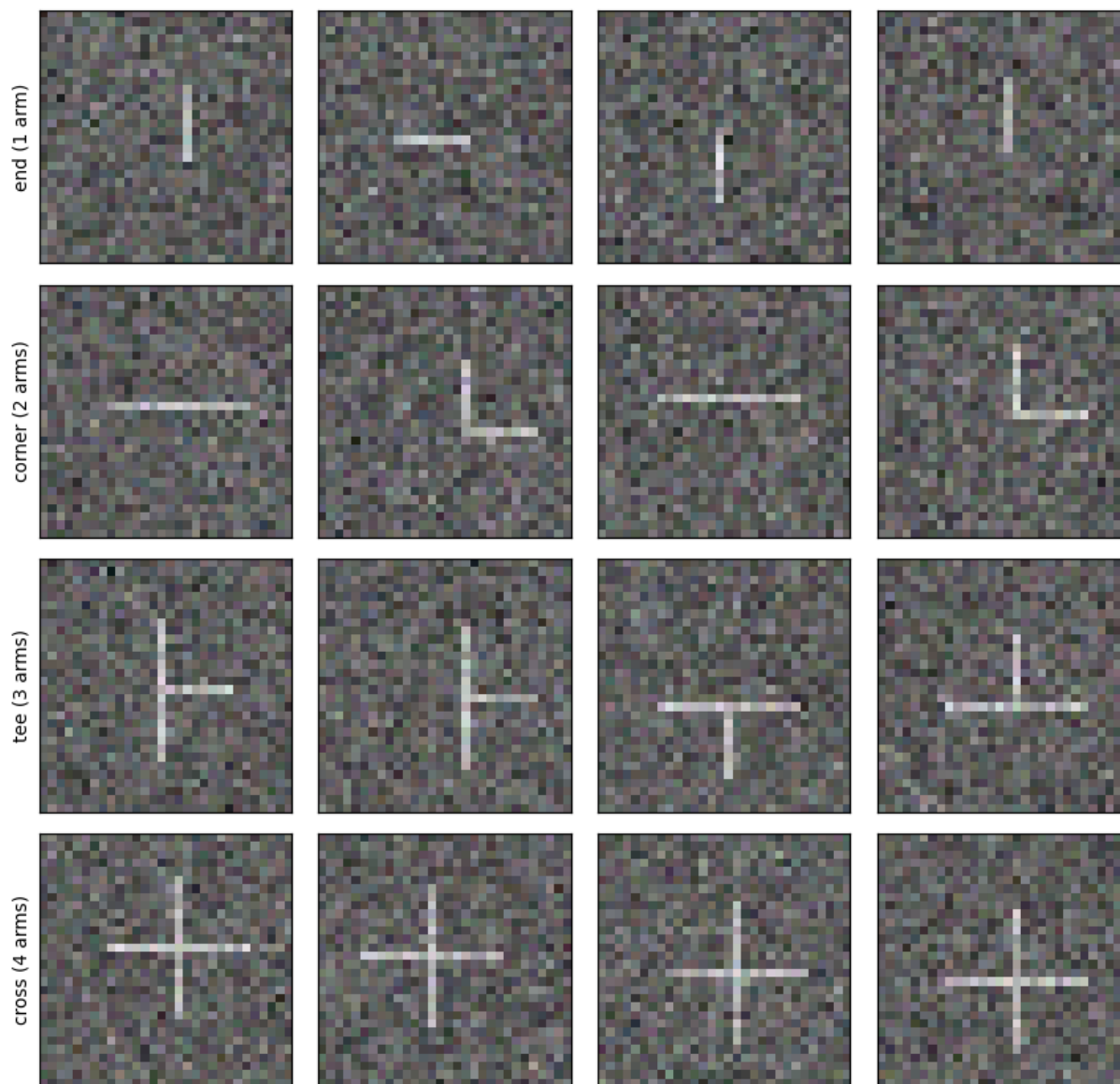
$rgb[c] = shape_signal + noise_c$ *noise_c drawn independently for $c \in \{R, G, B\}$*

```
rgb = np.stack([im, im, im], axis=0)
rgb = rgb + rng.randn(3, size, size) * channel_noise
```

np.stack([im, im, im], axis=0) replicates the SAME 2D array three times along a NEW axis 0, producing a (3, size, size) array where every channel starts identical. Adding **rng.randn(3, size, size) * channel_noise** — a FRESH random array, independently sampled per channel — is what then makes R, G, and B genuinely different from each other, not just three copies of one grayscale image.

REAL GRAPH (script's own output)

Colored junction dataset: same shapes as Day 42, now RGB with per-channel noise



REAL GRAPH (script's own output) -- 16 real example images from the colored junction dataset, 4 per class.

Segment 8 — OneBlockConvNetRGB / TwoBlockConvNetRGB / ThreeBlockConvNetRGB

```
class OneBlockConvNetRGB:
    def __init__(self, f1=3, k=3, img_size=30, hidden=8,
                  n_classes=4, lr=0.05, seed=42):
        self.Wc1 = rng.randn(f1, 3, k, k) * he_scale(3*k*k)
```

rng.randn(f1, 3, k, k) — the **3** here (not **1**, as Day 42's grayscale OneBlockConvNet used) is $C_{in}=3$ for a real RGB input. **he_scale(3*k*k)** matches: He initialization's fan-in must include ALL input channels the first conv layer actually sums over, not just spatial kernel size.

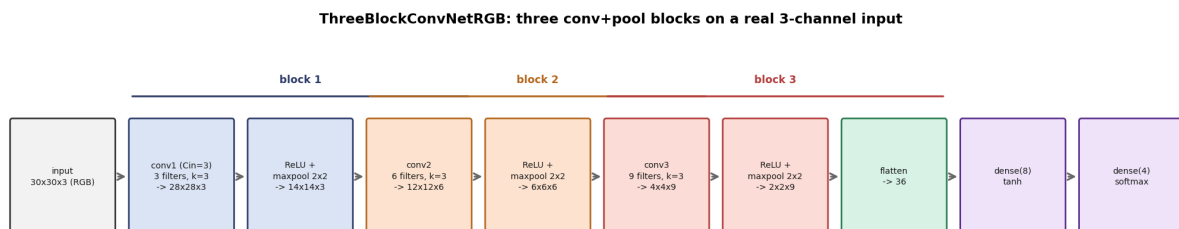
```
def forward(self, X):
    m = X.shape[0]
    self.X = X
    self.Zc1 = conv2d_forward_mc(self.X, self.Wc1, self.bc1)
```

self.X = X — notice there is NO **[:, None, :, :]** here, unlike Day 42's grayscale networks. Today's **X** already arrives shaped (m, 3, H, W) straight out of **make_junction_dataset_rgb**, so the channel axis already exists and needs no manual insertion.

```
class ThreeBlockConvNetRGB:
    def __init__(self, f1=3, f2=6, f3=9, k=3, img_size=30,
                  hidden=8, n_classes=4, lr=0.1, seed=42):
        ...
        self.Wc3 = rng.randn(f3, f2, k, k) * he_scale(f2*k*k)
        ...
        out3 = pool2 - k + 1
        pool3 = out3 // 2
        flat_dim = f3 * pool3 * pool3
```

The THIRD block's constructor is a direct copy-paste of the second block's pattern, renumbered: **Wc3** takes **f2** as its C_{in} (block 2's output channel count), and **out3/pool3** chain from **pool2** exactly as **out2/pool2** chained from **pool1** in Day 42's TwoBlockConvNet.

SYSTEM DIAGRAM



ThreeBlockConvNetRGB end to end: three conv+pool blocks on a real 3-channel input, with the exact shape at every stage.

Segment 9 — demo_three_block_gradcheck

```
net3 = ThreeBlockConvNetRGB(f1=3, f2=6, f3=9, k=3,
                             img_size=30, hidden=8, seed=1)
probs3 = net3.forward(X)
dz2 = probs3 - y_onehot
```

A fresh network instance (**net3**) with a fixed **seed=1** is built specifically for this check, separate from the one used for actual training later — isolating the gradient check from any state left over by other demos.

MATH

numgrad = (loss(Wc1+ε) – loss(Wc1)) / ε, ε=1e-5, compared to analytical dWc1

```
net4 = ThreeBlockConvNetRGB(f1=3, f2=6, f3=9, k=3,
                             img_size=30, hidden=8, seed=1)
net4.Wc1[i, j, k_, l] += eps
probs4 = net4.forward(X)
loss_plus = cross_entropy_loss(probs4, y_onehot)
numgrad = (loss_plus - loss_base) / eps
```

k_ (with a trailing underscore) is used instead of **k** here specifically because **k** is already the network's KERNEL SIZE parameter in scope — a small but real naming collision avoided by renaming the loop index, not the kernel size.

Segment 10 — demo_pooling_budget

```
size = 30
for i in range(1, 4):
    conv_out = cur - 3 + 1
    pool_out = conv_out // 2
    stages.append((i, cur, conv_out, pool_out))
    cur = pool_out
```

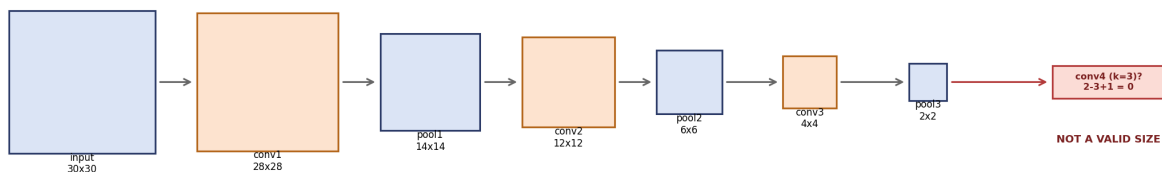
A plain loop recomputes each block's shrinkage arithmetic directly, rather than hardcoding the three blocks' sizes as literal numbers — so this section is genuinely **COMPUTING** the pooling budget, not just printing pre-known values.

```
try:
    X4 = rng.randn(1, 9, final, final)
    W4 = rng.randn(12, 9, 3, 3)
    Z4 = conv2d_forward_mc(X4, W4, b4)
    print(f"...returned shape {Z4.shape}")
except Exception as e:
    print(f"...confirms it: {type(e).__name__}: {e}")
```

The **try/except** here is almost a formality — **conv2d_forward_mc** does NOT raise an exception on a 2x2 input with a 3x3 kernel; Python's slicing and NumPy's array creation both silently accept **out_H = out_W = 0**. The **try** block actually executes the **print** inside it, confirming the function ran and returned a real, if useless, (1,12,0,0) array.

SYSTEM DIAGRAM

The pooling budget on a 30x30 input: three blocks fit, a fourth does not (no padding)



The pooling budget on a 30x30 input, visualized: three blocks fit, shrinking to 2x2; a fourth conv(k=3) cannot slide across it even once.

Segment 11 — demo_training, note_on_pytorch, and main()

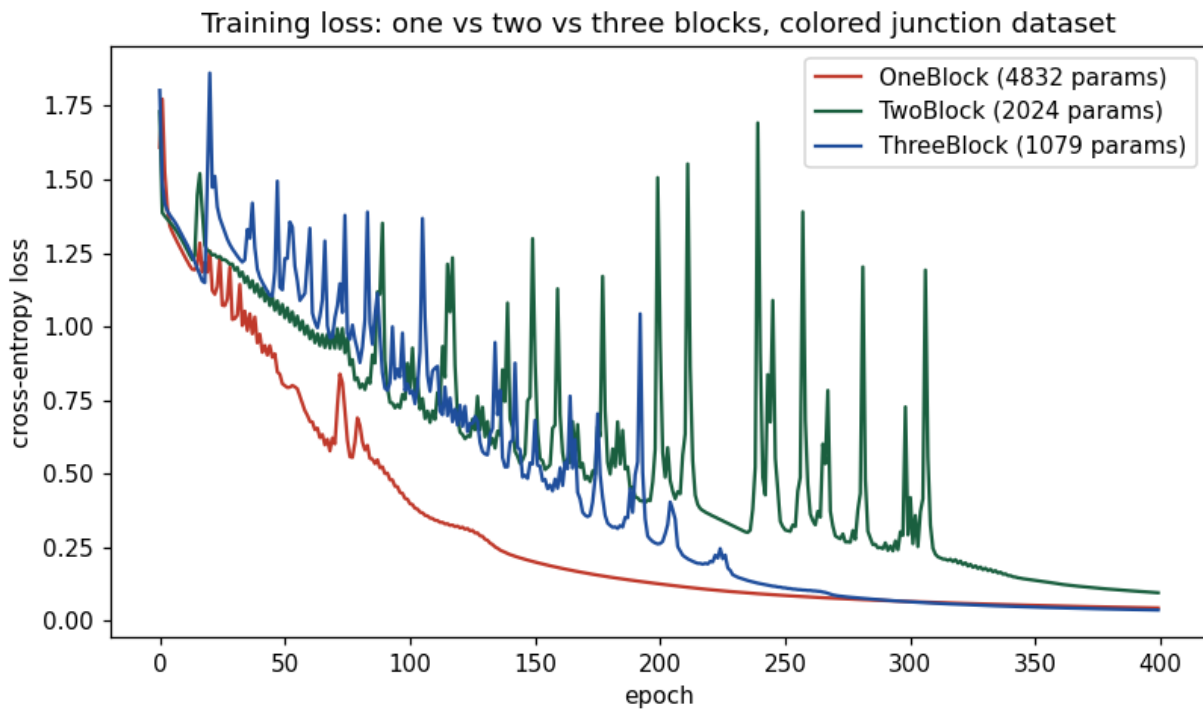
```
results = {}
net1 = OneBlockConvNetRGB(f1=3, k=3, img_size=30, hidden=8,
                           n_classes=4, lr=0.05, seed=42)
losses1 = net1.train(X, y, epochs=epochs)
results["one"] = dict(params=p1, test_acc=te1, ...)
```

Each of the three networks is trained in its own block with its own tuned learning rate (0.05, 0.08, 0.1) — all THREE reused the exact same **.train(X, y, epochs=epochs)** method signature, since every architecture implements the identical **forward/backward/train/accuracy** interface, just with a different number of blocks inside **forward/backward**.

```
reduction and accuracy printed via an f-string HEADLINE:
f"params {p1} -> {p2} -> {p3} fell steadily (monotonic), "
f"but test accuracy went {te1:.3f} -> {te2:.3f} -> "
f"{te3:.3f} -- NOT monotonic. ..."
```

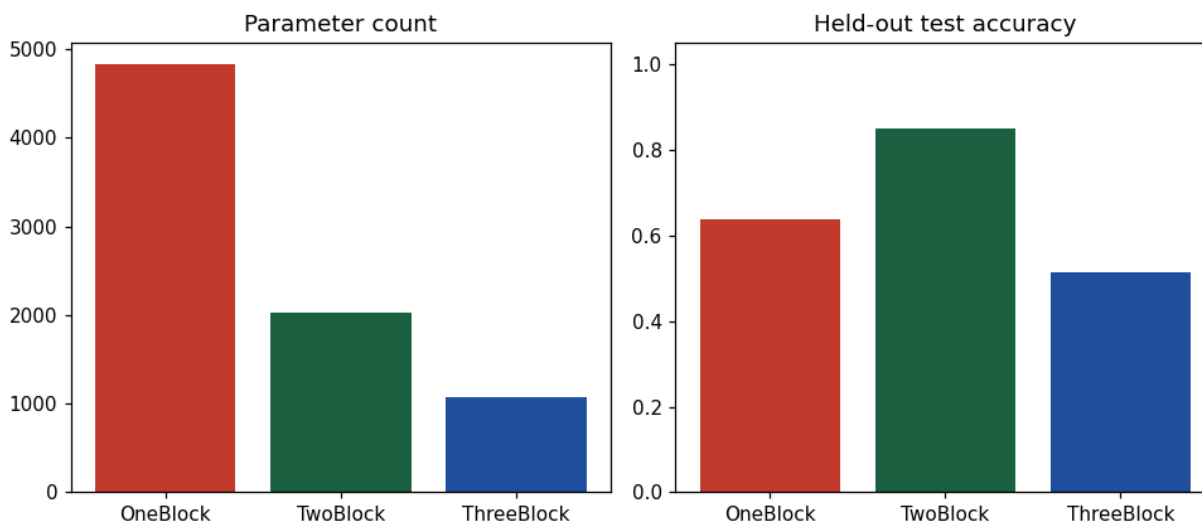
This lesson's headline sentence is built entirely from LIVE variables (**p1, p2, p3, te1, te2, te3**) computed earlier in the same run — if the real numbers had come out differently (say, monotonically improving), this exact same f-string would have printed a different, still-honest, sentence.

REAL GRAPH (script's own output)



REAL GRAPH (script's own output) -- training loss for all three networks, including real training instability.

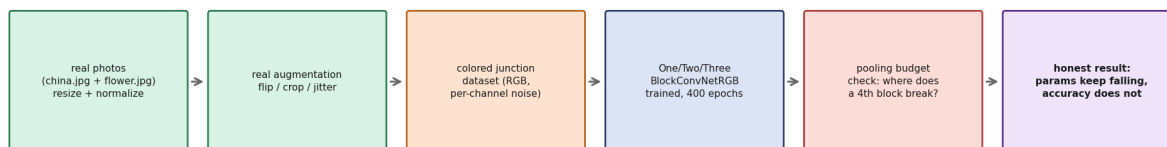
REAL GRAPH (script's own output)



REAL GRAPH (script's own output) -- parameter count falling steadily, held-out test accuracy rising then falling.

SYSTEM DIAGRAM

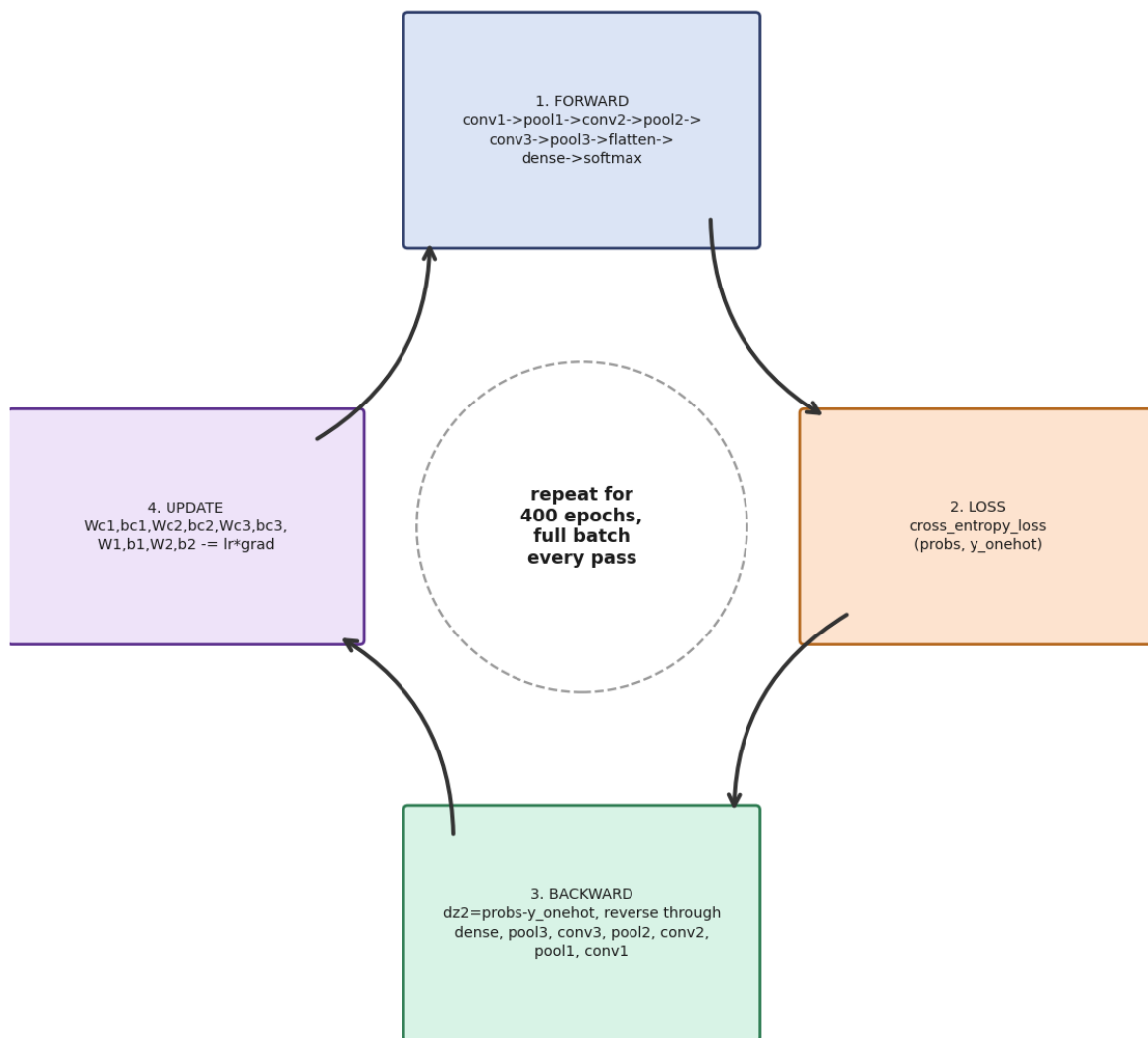
Day 43 pipeline: from two real photographs to an honest three-block comparison



The full Day 43 pipeline, start to finish: real photos to an honest three-block comparison.

SYSTEM DIAGRAM

The training work cycle, extended for THREE stacked conv+pool blocks



The training work cycle, extended for THREE stacked conv+pool blocks.

```
def main():
    china_r, flower_r, ... = demo_real_preprocessing()
    demo_augmentation(china_r)
    demo_receptive_field_three()
    X, y, ch_corr = demo_colored_dataset()
    demo_three_block_gradcheck()
    demo_pooling_budget()
    demo_training(X, y)
    note_on_pytorch()
```

main() runs every segment in this document's exact order — the gradient check (Segment 9) and pooling budget check (Segment 10) both run BEFORE the real training comparison (Segment 11), so by the time any network is actually trained, its correctness and the depth limit it might approach are both already established.

Quick Syntax Glossary — New or Reinforced Constructs

`Image.fromarray(...).resize(..., Image.LANCZOS)` — Converts a NumPy array to a PIL image and resizes it with a high-quality resampling filter -- the standard-library way real pipelines resize photos, used instead of hand-rolled interpolation.

`img.mean(axis=(0, 1), keepdims=True)` — Averages over height and width only, leaving the channel axis untouched -- produces one mean per channel (per-channel normalization) instead of one mean for the whole image.

`array[:, ::-1, :]` (negative-step slicing) — Reverses an axis by slicing with a step of -1 -- here, reversing the width axis performs a horizontal flip with no dedicated flip function needed.

`np.clip(x, 0, 255)` — Clamps every value in an array to a valid range -- essential after brightness jitter, since multiplying pixel values can push them above 255, an invalid pixel intensity.

`Xp[0, :, r, c] += eps` (colon in a specific axis position) — Perturbs ALL channels at one spatial position simultaneously -- treats a 'pixel' as one unit spanning R, G, B together for the receptive-field sensitivity sweep.

`np.stack([im, im, im], axis=0)` — Stacks three separate 2D arrays along a brand-new axis 0, producing a (3, H, W) array -- turns one grayscale array into a 3-channel starting point before per-channel noise is added.

`center_jitter + 1` in `rng.randint`'s upper bound — `randint`'s upper bound is exclusive, so reaching the true maximum jitter value requires adding 1 -- an easy off-by-one to miss when converting a fixed range into a jitter amount.

`try/except` around a call that doesn't actually raise — `conv2d_forward_mc` accepts a 2x2 input with a 3x3 kernel without error, returning a (1,12,0,0) array instead -- the try block here documents that the failure is silent and shape-based, not an exception.

f-string headline built from live variables — The training comparison's summary sentence embeds `p1/p2/p3/te1/te2/te3` directly, so it always reflects whatever numbers THIS run actually produced, not a fixed narrative.

End of Day 43 syntax reference. For the real-image, receptive-field, and pooling-budget concepts these lines implement, see the main Day 43 study guide PDF.